

Applying SG/Send's Open Source Strategy to StackGuard

Prepared by: Ambassador + Alchemist, SGraph Send Team

Architect review: PKI mapping grounded in v0.13.12 reality

Date: 10 March 2026

Context: Strategic analysis for Dinis Cruz's conversation with StackGuard's founder

Opening Frame

SG/Send made a deliberate, early-stage decision to go Apache 2.0. Not as a marketing tactic — as an architectural statement: **for a zero-knowledge security product, open source is not optional**. A security reviewer put it plainly: *"Anyone can verify" is not a nice-to-have. It's the product.*

StackGuard is solving the NHI problem from the detection and remediation side. The same logic applies. You cannot ask enterprises to trust a black box with their most sensitive infrastructure inventory — their secrets, service accounts, tokens, and certificates. The tool governing NHIs needs to be as trustworthy as the identities it governs.

This document maps SG/Send's open source strategy, pillar by pillar, and shows how each one translates to StackGuard's context.

1. The Core Insight: Trust Is the Product

How SG/Send Thinks About It

SG/Send's Apache 2.0 licence is load-bearing. It does four things simultaneously:

1. **Verifiable security claims** — anyone can audit the encryption implementation. For zero-knowledge crypto, this is the only way to make the claim credible.
2. **Frictionless enterprise procurement** — no vendor lock-in objection, no legal review delay, no InfoSec team veto on a closed-source security tool.
3. **Community trust** → **enterprise demand** — the Redis, Elastic, Confluent model. Open-source adoption at the developer/practitioner level drives enterprise sales from the bottom up.
4. **Terminal value** — even in a worst case, the technology has community value and cannot be killed by a single company's failure.

Applied to StackGuard

StackGuard is asking enterprises to hand over something extraordinarily sensitive: a complete inventory of every NHI, every secret, every service account, every misconfiguration in their environment. That is a level of access that makes a typical SaaS integration look trivial.

The enterprise procurement question is not "does StackGuard work?" It is: **"can we trust StackGuard with the keys to the kingdom?"**

Open source doesn't just answer that question. It makes the question answerable — provably, auditably, permanently.

The Alchemist's translation: Closed-source NHI governance is a contradiction in terms. You are asking enterprises to solve their secrets management problem by handing their secrets to a black box. Open source removes the contradiction.

2. The Architectural Split: What to Open, What to Keep

SG/Send's model is instructive here. The entire core is open source. The commercial moat is not in code secrecy — it's in the architecture being *not retrofittable* by competitors. Zero-knowledge + PKI + self-hostable is a systems design property, not a trade secret.

StackGuard should think the same way. The question is where the split sits.

Proposed Open/Closed Architecture for StackGuard

Layer	Open Source	Commercial / Managed
Detection engine — the scanning logic, regex patterns, connector protocol	✔ Open (Apache 2.0)	—
Connector library — integrations for GitHub, AWS, GCP, Okta, Kubernetes, etc.	✔ Open (community-extensible)	—
NHI schema and taxonomy — the model for what an NHI is, its lifecycle states, risk classifications	✔ Open (becomes the standard)	—
Remediation playbooks — the logic for how to rotate, vault, revoke	✔ Open (base playbooks)	Commercial (AI-driven, custom playbooks, enterprise workflows)
Governance engine — the policy, compliance mapping, CI gating	Partial	Commercial (SOC 2, ISO 27001, custom policy)
Analytics and intelligence — risk scoring, threat modelling, cross-environment correlation	—	Commercial (SaaS, managed service)
Managed cloud service — the hosted platform, support, SLAs	—	Commercial

The strategic logic: Open source the scanning and detection layer. This is where community contributions have the most leverage — more integrations, more pattern coverage, faster response to new secret formats. Keep the intelligence, governance, and managed service commercial. The same Redis/Elastic/Confluent playbook.

3. Pillar by Pillar: SG/Send Strategy Applied

Pillar 1: "Verify It Yourself" Beats "Trust Us"

SG/Send's version: The zero-knowledge guarantee is provable by reading the code. No one has to take our word for it. A security community that can audit the crypto is more valuable than any certification.

StackGuard's version: The NHI scanner that reads your AWS credentials, your GitHub tokens, your Kubernetes service accounts — make that auditable. Security teams at enterprises already do code reviews of their tooling. An open-source scanner passes that review in a way a closed binary never will.

The competitive framing: Tresorit (closed-source, encrypted storage) loses enterprise procurement battles to SG/Send on this dimension alone. StackGuard open-sourcing its scanner creates the same asymmetry against any NHI competitor that stays closed.

Pillar 2: Frictionless Enterprise Procurement

SG/Send's version: Apache 2.0 means no legal review, no vendor lock-in objection, no InfoSec veto. The procurement conversation skips a step.

StackGuard's version: Enterprise security teams are among the most procurement-friction-aware buyers in any market. An open-source core means: legal doesn't need to review it, InfoSec can audit it themselves, and procurement can't be blocked on "we don't allow third-party access to our secrets management data." The closed-source sales cycle at an enterprise with a serious security posture is brutally long. Open source shortens it structurally.

One additional dimension unique to NHI: **regulated industries** (finance, healthcare, defence) often cannot use closed-source tools for certain compliance-sensitive functions. Open source unlocks those customers by default.

Pillar 3: Community Coverage as Competitive Moat

SG/Send's version: Every contributor to the encryption library or integration adds to the coverage and credibility. Community audit is a marketing and quality function simultaneously.

StackGuard's version: StackGuard's value scales with its integration coverage. Every new connector (a new CI/CD platform, a new cloud provider, a new SaaS tool) increases the comprehensiveness of the NHI inventory. Closed source means StackGuard's team builds every connector. Open source means the community that uses Gitea, Hugging Face, JFrog, and every other platform in StackGuard's existing list contributes the connectors they need. The community builds the long tail. StackGuard maintains the core.

This is also a moat: a community-built connector library is harder to replicate than a proprietary one. A competitor can clone a closed-source scanner. They cannot clone three years of community contributions overnight.

Pillar 4: Terminal Value

SG/Send's version: Even in a worst case, Apache 2.0 means the technology has community value. The project can be forked, maintained, or acquired.

StackGuard's version: Enterprise customers making long-term bets on StackGuard need to believe it will still exist in five years. Open source provides the insurance policy: even if StackGuard-the-company pivots or fails, the NHI scanning capability lives on. This is an explicit procurement objection handler in large enterprise deals ("what happens to our data and tooling if you get acquired or shut down?").

Pillar 5: Security Community Endorsement as Distribution

SG/Send's version: A CISO's endorsement is worth more than a million impressions. We earn credibility by being correct, not by being loud. The security community validates us by reading the code.

StackGuard's version: The NHI security community — red teamers, AppSec practitioners, DevSecOps engineers — is exactly the audience that discovers StackGuard exists. They find it at conferences, on GitHub, in blog posts. If the scanner is open source, they star it, fork it, contribute to it, and recommend it to their employers. Closed source means that same community evaluates it with scepticism, or doesn't engage at all. The security community trusts tools it can read.

4. The Runs-Everywhere Angle

SG/Send's "runs-everywhere" strategy (Lambda, Docker, Fargate, GCP Cloud Run, EC2, CLI, PyPI) is enabled by its open source architecture. The same application code deploys to seven targets because there are no proprietary dependencies or black-box services embedded in the core.

For StackGuard, this translates directly:

Customer Requirement	StackGuard Response (with open source)
"We need on-premise scanning — nothing leaves our environment"	Open-source scanner deploys on bare metal / private cloud
"We're in a regulated industry — no SaaS for our secrets inventory"	Self-hosted deployment, auditable code
"We need to integrate scanning into our own CI/CD pipeline"	Open-source CLI/SDK, community-supported
"We're air-gapped / classified environment"	Fully self-contained open-source deployment
"We use Kubernetes everywhere"	Community-contributed K8s operator

The commercial managed service still exists for everyone else. But the open-source path removes the "we can't use SaaS for this" blocker that is common in StackGuard's most valuable customer segments (finance, defence, healthcare, government).

5. The SG/Send PKI Connection: Open Source as Trust Infrastructure

This is where the Ambassador, Alchemist, and Architect perspectives converge — and where the most interesting conversation with StackGuard lives.

SG/Send's proposed identity infrastructure (`sgraph.ai/verify`), (`.well-known/sg-identity.json`), authentication by decryption, signed communications) is designed to be **open, verifiable, and community-governed**. Not because it's politically convenient — because cryptographic identity infrastructure that isn't open is just another secret waiting to be compromised.

If StackGuard adopts a PKI-based NHI identity model (as the Architect's analysis suggests is the direction of travel), that model needs to be open source for the same reason. The NHI identity schema, the revocation protocol, the transparency log format — these are infrastructure primitives that the industry will need to adopt broadly. Proprietary infrastructure primitives don't get adopted broadly. Open ones do.

The precedent: Certificate Transparency (the protocol for auditing TLS certificates) is open. SPIFFE/SPIRE (the CNCF standard for workload identity) is open. DNS (the backbone of internet identity) is open. StackGuard has the opportunity to define the NHI identity standard in the same way — and that only works if it's open.

The Alchemist's framing for investors: "We are not building a product. We are building the standard. Open source is how standards get adopted. Commercial services are how standards generate revenue. The Redis/Elastic/Confluent model, applied to the NHI identity problem."

6. What This Looks Like in Practice

Immediate Actions

1. **Open-source the scanner core** on Apache 2.0 or MIT. Keep governance, analytics, and managed service commercial. Announce with a clear architectural rationale — not "we're being generous" but "this is the right design for a tool this enterprises need to trust."
2. **Publish the NHI schema** as an open specification. Invite the community to extend it. Position StackGuard as the steward of the standard, not the owner of the data model.
3. **Create a CLI/SDK** (open source) that security teams can embed in their own pipelines. This is SG/Send's PyPI play — the distribution multiplier that turns a web tool into a platform.
4. **Engage the CNCF / OWASP ecosystem.** SG/Send's founder has OWASP board history. The NHI problem deserves a working group. StackGuard could seed it and own the conversation.

Medium-Term

5. **Partner on identity infrastructure.** SG/Send's PKI key registry and transparency log are open source. If StackGuard's NHI identity model uses the same primitives, the integration story becomes: SG/Send provides the cryptographic identity layer; StackGuard provides governance and lifecycle management over those identities. Two open-source systems composing to solve the full problem.
6. **Bug bounty program.** SG/Send's marketing strategy explicitly includes a developer community budget for bug bounty seeding. For StackGuard, a bug bounty on the open-source scanner is direct credibility — it demonstrates confidence in the code and the community's ability to validate it.

7. The One-Slide Version

	Closed Source	Open Source
Enterprise trust	"Trust us"	"Verify it yourself"

	Closed Source	Open Source
Procurement friction	Legal review, InfoSec veto	Apache 2.0 passes review by default
Regulated industries	Blocked (no SaaS for secrets data)	Unblocked (self-hosted, auditable)
Community coverage	StackGuard team builds every connector	Community builds the long tail
Standard setting	Product feature	Industry standard
Competitive moat	Code secrecy (fragile)	Architecture + community + data (durable)
Terminal value	Zero if company fails	Lives on as community project

8. The Conversation Opener for the Meeting

"You're asking enterprises to solve their secrets problem by trusting a black box with their secrets. We solved the same trust problem in a different domain — zero-knowledge file transfer — and the answer was the same: open source isn't the product strategy, it's the prerequisite. Here's how we think about the split between what you open and what you keep."

SGraph Send — Ambassador + Alchemist
Architect-reviewed against v0.13.12 reality
10 March 2026